

Reviewer Pack — Minimal Number of Generators for Affine Plane Curves and AC0...

Entry 17c4e5e49704 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 22:07:45 UTC

Minimal Number of Generators for Affine Plane Curves and AC0 Circuit Lower Bounds

Entry ID: 17c4e5e49704 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 22:07:45 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): AC⁰ Circuit Complexity

Statement:

For any boolean function f with n variables, the minimal number of generators of its affine plane curve representation is lower-bounded by $2^{n/4}$.

Rationale (proposer's reasoning):

The generators of an affine plane curve correspond to its points in projective space. The structure of these points provides a geometric intuition for complexity classes like AC⁰. A direct correspondence between the number of generators and circuit size could reveal hidden structures in both fields.

Taxonomy category: Affine_Geometry_x_AC0_Circuit_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d52167486a28f9aa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the average minimum number of generators across all randomly generated boolean functions is $\geq 2^{n/4}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine geometry" AND "AC0 circuit complexity" AND curve representation" - "minimal number of generators" AND "affine plane curve" AND AC0" - "boolean function" AND " $2^{n/4}$ lower bound" AND "affine geometry"

Top relevant hits considered: - [s2:2605.01637] The Banach-Butterfly Invariant: Influence-Adaptive Walsh Geometry for Ternary Polynomial Threshold Functions - [s2:10.1007/s12095-020-00445-z] Boolean Functions with Multiplicative Complexity 3 and 4

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def count_generators(f):
        n = len(f)
        points = set()
        for i in range(2**n):
            point = tuple(f[i >> j & 1] for j in range(n))
            if all(point[j] == f[i ^ (1 << j)] for j in range(n)):
                points.add(point)
        return len(points)

    n_values = [5, 10, 15, 20, 30, 40]
    total_generators = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):
            f = generate_boolean_function(n)
            generators = count_generators(f)
            if generators < 0: # Ensure non-negative number of generators
                return {
                    "metric_name": "min_generators",
                    "metric_value": None,
                    "instances_tested": instances_tested,
                    "conjecture_holds": False,
```

```

        "counterexample": "negative_generators"
    }
    total_generators += generators
    instances_tested += 1

mean_generators = Fraction(total_generators, instances_tested)
conjecture_holds = mean_generators >= Fraction(2**n_values[-1], 4)

return {
    "metric_name": "min_generators",
    "metric_value": float(mean_generators),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={seeds[first_failing_seed]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_184da865.py", line 68, in <module>
    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_184da865.py", line 40, in run_trial
    generators = count_generators(f)
    ^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_184da865.py", line 29, in count_generators
    if all(point[j] == f[i ^ (1 << j)] for j in range(n)):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_184da865.py", line 29, in <genexpr>
    if all(point[j] == f[i ^ (1 << j)] for j in range(n)):

```

~ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify the conjecture's claim. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12734
2	preregistration	ollama_remote	glm4:latest	0	0	11950
3	novelty	ollama_remote	glm4:latest	0	0	8147
4	novelty	ollama_remote	glm4:latest	0	0	10014
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11927
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8259
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12124
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8378
9	judge	ollama_remote	glm4:latest	0	0	13999

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97531 ms total latency. Provider mix: {'ollama remote': 9}

(full prompt+response transcripts available in [research/audit/17c4e5e49704.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/17c4e5e49704.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp sandbox/test *.py (per-cycle)

Replay tarball: `research/replay/17c4e5e49704.tar.gz` (if generated)